running that has a cost. In the serverless model, however, resources are provisioned only when necessary. If there is nothing to process, everything is shut down. That's what we call *scaling to zero*, and it's one of the main features offered by serverless platforms.

A consequence of scaling applications to zero is that when eventually there's a request to handle, a new application instance is started, and it must be ready to process the request very quickly. Standard JVM applications are not suitable for serverless applications, since it's hard to achieve a startup time lower than a few seconds. That's why GraalVM native images became popular. Their instant startup time and reduced memory consumption make them perfect for the serverless model. The *instant startup time* is required for scaling. The *reduced memory consumption* helps reduce costs, which is one of the goals of serverless and cloud native in general.

Besides cost optimization, serverless technologies also move some extra responsibility from the application to the platform. That might be an advantage, since it allows developers to focus exclusively on the business logic. But it's also essential to consider what degree of control you would like to have and how you will deal with vendor lock-in. Each serverless platform has its own features and APIs. Once you start writing functions for a specific platform, you can't move them easily to another, as you would do with containers. You might compromise to gain responsibility and scope and lose on control and portability more than with any other approach. That's why *Knative* became popular quickly: it's built on Kubernetes, which means that you can easily move your serverless workloads between platforms and vendors.

This section will guide you through developing and deploying a serverless application. You'll use Spring Native to compile it to a GraalVM native image and use Spring Cloud Function to implement the business logic as